

R Code Examples from WDDA Lecture 01

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-04-29

Contents

1	Introduction	1
2	Importing a Dataset	1
3	R Syntax Styles	2
4	Navigating Data Frames	2
5	Using <code>attach()</code> and <code>detach()</code>	3
6	Using <code>length()</code>	3

1 Introduction

This document collects all R code examples from the WDDA Lecture 01 slides. It contains additional comments, explanations, and examples to make the individual code snippets easier to understand.

2 Importing a Dataset

In this section we import a dataset using the **readxl** package. `read_excel()` automatically detects column types. Use `str()` to inspect the structure of the imported data.

```
library(readxl) # Load the readxl package for importing Excel files
markets <- read_excel("WDDA_01.xlsx", sheet = "Exchanges") # Import the data from the
  ↪ "Exchanges" sheet
```

Additional tip: Inspect the structure of the imported data frame:

```
str(markets) # Show the structure of the data frame 'markets'
```

```
## tibble [21 x 3] (S3: tbl_df/tbl/data.frame)
## $ exchange: chr [1:21] "Athens" "Borsa Italiana" "Bratislava" "Bucharest" ...
## $ trades : num [1:21] 599192 5921099 111 79921 298871 ...
## $ turnover: num [1:21] 2009.8 44385.9 0.1 45.3 1089.6 ...
```

Note: R stores the object `markets` as a data frame.

3 R Syntax Styles

R offers several ways of writing code. The two main styles are:

1. **Classic (nested) syntax:** functions are read from the inside out
2. **Pipe syntax:** functions are read from left to right (like a pipeline)

Here is a comparison of the two styles:

```
# Classic syntax (nested)
mean(sqrt(abs(x))) # Read from inside to outside: first abs(), then sqrt(), then mean()

# Pipe syntax with |>
x |> abs() |> sqrt() |> mean() # Read left to right: first abs(), then sqrt(), then
  ↪ mean()

# A more complex example:
# Classic
subset(transform(data, z = x + y), z > 0)

# With the pipe
data |>
  transform(z = x + y) |>
  subset(z > 0)
```

The pipe syntax (`|>`) was introduced in R 4.1.0 and is now the recommended style for readable code. The older pipe operator (`%>%`) from the `magrittr` package works similarly.

Tip: In RStudio you can use the keyboard shortcut **Ctrl + Shift + M** to insert the pipe operator. To switch between the native (`|>`) and the `magrittr` (`%>%`) pipe operator, go to: Tools -> Global Options -> Code -> “Use native pipe operator” (Source: Stack Overflow)

4 Navigating Data Frames

Here we show different methods for inspecting and subsetting a data frame. Each method is useful for different tasks.

```
# Show the entire data frame
markets

# Show the column names of the data frame
names(markets)

# Classic subsetting examples:
markets$trades # Access the column 'trades'
markets[3,]    # Access the third row
markets[,2]    # Access the second column
markets[3,2]   # Access the element in the 3rd row and 2nd column

# Using tidyverse functions for a more readable syntax:
markets |> pull(trades) # Extract the column 'trades' as a vector
markets |> slice(3)     # Access the third row
markets |> select(2)    # Select the second column by position
```

```
markets |> select(trades)      # Select the column 'trades' by name
markets |> slice(2) |> select(trades) # First take the second row, then select the
  ↪ column 'trades'
```

Additional example: Show the first six rows of the data frame:

```
head(markets) # Show the first six rows of the data frame
```

```
## # A tibble: 6 x 3
##   exchange      trades turnover
##   <chr>         <dbl>     <dbl>
## 1 Athens      599192    2010.
## 2 Borsa Italiana 5921099  44386.
## 3 Bratislava    111        0.1
## 4 Bucharest    79921     45.3
## 5 Budapest    298871    1090.
## 6 Bulgarian    14040     64.4
```

5 Using attach() and detach()

The function `attach()` makes it easier to access the variables of a data frame by adding them to the search path. However, this can lead to naming conflicts if multiple objects share the same name. An alternative is `with()`, which restricts the scope.

```
# Attach the data frame so that its columns can be accessed directly
attach(markets)

# Now 'trades' can be accessed directly, without prefixing the data frame name
trades

# Remove the data frame from the search path to avoid naming conflicts
detach(markets)

# After detaching, accessing 'trades' without the data frame name will throw an error:
trades
```

Additional note: An alternative to `attach()` is `with()`, which only takes effect inside a specific expression:

```
with(markets, summary(trades)) # Summarise the column 'trades' without permanently
  ↪ attaching the data frame
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      111   31167  433545 2710837 2799329 16539588
```

6 Using length()

The function `length()` behaves differently for data frames and vectors. For a data frame, `length()` returns the number of columns (variables), not the number of rows (observations). To count rows, use `nrow()`.

```
# For a data frame, length() returns the number of columns
markets |> length()

# For a vector, length() returns the number of elements (observations)
trades |> length()

# To get the number of rows (observations) of a data frame, use nrow():
nrow(markets)
```

Additional tip: You can use `ncol()` to obtain the number of columns:

```
ncol(markets)
```

```
## [1] 3
```